AI Bootcamp — Week 2: Software Development Workflow

Thursday — Student Exercise Guide

© 2026 Improving

OBJECTIVES

By end of Week 2, the candidate can:

- Apply the EDD discipline to a domain with right/wrong answers: software development artifacts
- Build out the DLP student-workspace skeleton skills through EC-first iteration, adding Context and Constraints that are demonstrably load-bearing
- Explain the pipeline's YAML file contracts: what each skill reads and writes, the fail-fast behavior on missing inputs, and the review write-back loop
- Direct the AI to write a Python evaluation script against their EC (deterministic checks first, LLM judge second) and run baseline + iteration cycles with it
- Trace quality degradation through a multi-stage pipeline and isolate the source
- Produce an evaluation-measured iteration history showing net improvement across at least one full skill

© 2026 Improving

TIMING

- **Monday–Friday:** 9:00 AM – 5:00 PM with 1-hour lunch (7 hours each day)

© 2026 Improving

THURSDAY (9:00 AM – 5:00 PM): CREATE EPICS & STORIES AND PLAN STORY

9:00 – 10:30 AM: WRITE EC FOR EPICS & STORIES AND PLAN STORY

`create-epics-stories` takes three upstream outputs as input (PRD + Architecture + UX) and writes `stories.yaml`. Before writing EC, consider: what are the most common ways story decomposition fails?

- Stories that are too large to implement in a sprint
- Acceptance Criteria that are not testable
- Stories that omit dependencies
- Stories that duplicate work described in other stories
- AC that prescribe implementation detail (remember the AI-generated sample stories)

Define EC that catch these specific failure modes. Note that the schema gives you free deterministic checks: 3–8 AC per story, at least one negative or edge_case AC per story, unique story IDs. Those run without any model call.

10:30 AM – 12:00 PM: CREATE EPICS & STORIES: BUILD AND ITERATE

EDD cycle for `create-epics-stories`. Because this skill takes rich upstream context as input, its Context section should describe the intent: how to interpret and prioritize information from the three upstream documents when they conflict or are incomplete.

12:00 – 1:00 PM: LUNCH

1:00 – 3:00 PM: PLAN STORY: BUILD AND ITERATE

EDD cycle for `plan-story`.

The plan quality dependency, named explicitly: code is generated from the plan, never directly from the story. A vague plan produces vague code; a specific plan produces specific code. This is why `plan-story` fails fast if `stories.yaml` is missing, and why its schema requires every AC to be referenced by at least one task's `acceptance_criteria_refs`.

This skill is often where candidates first encounter the challenge of injecting technical specificity: a plan for a Node.js API looks different from a plan for a Python service. Context must carry the intent and the environment. Test that it is load-bearing by running the same story through the skill with two different stack descriptions; the plans should differ in ways that are specific to the stack.

3:00 – 5:00 PM: THREE-STAGE PIPELINE CHECK

Run a feature request through PRD → Architecture → UX → Epics & Stories → Plan Story. Review the `plan.yaml` output against the original feature request. Trace any quality gaps back to their source. Check the schema contract held: every AC referenced, tasks in dependency order.

© 2026 Improving

© 2026 Improving